

## # Interview Preparation Guide – Gaurav

### ## 1) Short Natural Self-Introduction (Use this in interview)

Hi, I'm Gaurav. I'm a software developer with hands-on experience in JavaScript-based development, especially with React on the frontend and Node.js on the backend. In my projects, I've worked with APIs, data handling, and real-world problem solving using practical coding approaches. I enjoy building clean and scalable solutions, and I focus on writing code that is easy to maintain and improve over time. Recently, I've been strengthening my understanding of TypeScript, design principles like SOLID, and microservices concepts so I can contribute better to larger production systems.

### ## 2) Most Important Topics (Priority)

#### ### MongoDB (Basics + Data Modeling)

- **What it is**: A NoSQL document database that stores JSON-like BSON documents.
- **Basic operations**:
  - `insertOne`, `insertMany`
  - `find`, `findOne`
  - `updateOne`, `updateMany`, `$set`, `$inc`
  - `deleteOne`, `deleteMany`
- **Data modeling**:
  - **Embedding**: Keep related data in same document (fast reads).
  - **Referencing**: Keep relations via IDs for large/independent data.
  - Use indexes on frequently queried fields.
  - Avoid unbounded array growth in one document.
- **Interview tip**: Explain *when* to embed vs reference with an example.

#### ### React Core Concepts

- **Components**: Reusable UI building blocks.
- **Hooks**:
  - `useState`: local state.
  - `useEffect`: side effects (API calls, subscriptions).
  - `useMemo` / `useCallback`: performance optimization.
- **State management**:
  - Local state (`useState`), shared state (Context API), large app state (Redux/Zustand).
- **Interview tip**: Mention one real bug you solved related to state/effect dependencies.

#### ### TypeScript

- **Why TypeScript over JavaScript**:
  - Static typing catches errors at compile time.
  - Better IDE support/autocomplete/refactoring.
  - Safer large-scale codebase maintenance.
- **Core concepts**:
  - `type` and `interface` for contracts.
  - Union/intersection types.
  - Generics for reusable typed logic.
- **Interview tip**: "TypeScript improves developer confidence and reduces runtime bugs."

#### ### SOLID Principles (Practical)

- **S**: Single Responsibility – one class/module, one reason to change.

- **O**: Open/Closed – open for extension, closed for modification.
- **L**: Liskov Substitution – child class should safely replace parent.
- **I**: Interface Segregation – smaller focused interfaces.
- **D**: Dependency Inversion – depend on abstractions, not concrete classes.
- **Interview tip**: Give one practical example from service/controller separation.

### ### Design Patterns (Singleton, Factory, MVC)

- **Singleton**: One instance shared globally (e.g., DB connection manager).
- **Factory**: Create objects via one method based on input type.
- **MVC**:
  - Model: data/business logic
  - View: UI response
  - Controller: request handling
- **Interview tip**: Explain why pattern improved maintainability.

### ### Microservices Architecture

- **Definition**: Application split into small independent services.
- **Communication**:
  - Sync: REST/gRPC
  - Async: Kafka/RabbitMQ events
- **Scalability**: Scale only high-traffic services.
- **Deployment**: Independent CI/CD per service, containerized deployment.
- **Challenges**: Distributed logging, tracing, eventual consistency.

---

### ## 3) JavaScript Important Questions (Quick Revision)

1. **Event Loop**: Handles async callbacks in single-threaded JS using call stack + callback/task queues.
2. **IIFE**: Immediately Invoked Function Expression. Used to create private scope and avoid global pollution.
3. **Callback vs Promise**:
  - Callback: function passed into another function.
  - Promise: cleaner async handling with `then/catch`, better chaining.
  - `Promise.all`: fails fast if one rejects.
  - `Promise.allSettled`: waits for all; returns each status.
4. **Hashing vs Encryption**:
  - Hashing: one-way, for passwords/integrity.
  - Encryption: reversible with key, for confidentiality.
5. **var vs let vs const**:
  - `var`: function scoped, hoisted with `undefined`.
  - `let`: block scoped, temporal dead zone.
  - `const`: block scoped, cannot reassign binding.
6. **Hoisting**: Declarations move to top conceptually; initialization does not.
7. **Closure**: Inner function remembers outer scope variables after outer function finishes.
8. **Call Stack**: Tracks function execution order (LIFO).
9. **bind vs call**:
  - `call`: invokes immediately with `this` + args.
  - `bind`: returns new function with fixed `this`.
10. **Scope Chain**: JS resolves variable from local to outer/global scope.
11. **Prototype**: Object inheritance mechanism in JS.

12. **try-catch-finally**: Exception handling; `finally`` runs always.

---

## ## 4) Node.js Important Questions (Quick Revision)

1. **What is Node.js**: JavaScript runtime built on V8, for server-side development.
2. **Latest version**: Mention LTS/current from official Node site at interview time.
3. **Advantages**: Fast I/O, same language full stack, huge npm ecosystem.
4. **Disadvantages**: CPU-heavy tasks can block event loop, callback complexity if poorly designed.
5. **Internal architecture**: Event loop + libuv + thread pool + V8.
6. **Built-in modules**: `fs``, `http``, `path``, `os``, `events``, `stream``, `crypto``, `child_process``.
7. **Express.js**: Minimal web framework for routes/middleware.
8. **Express methods**: `app.get``, `app.post``, `app.put``, `app.delete``, `app.use``.
9. **Event loop in Node**: Manages non-blocking async operations.
10. **Error handling**: `try/catch`` (sync), middleware/error callbacks (async), centralized handlers.
11. **Read file**: `fs.readFile`` / streams for large files.
12. **Multiple threads**: Worker Threads.
13. **Multiple processes**: `cluster`` or `child_process``.
14. **Global variables**: `global`` object (use carefully).
15. **NPM**: Package manager for install, scripts, publish.
16. **Nexus / package publish**: Host private npm packages and publish versions.
17. **CI/CD, Jenkins, API Gateway, OpenShift**:
  - CI/CD automates build-test-deploy.
  - Jenkins pipeline execution.
  - API Gateway as single entry/security/routing.
  - OpenShift for container orchestration.
18. **Kafka / RabbitMQ / ActiveMQ**:
  - Messaging/event streaming for decoupling services.
19. **Producer-Consumer architecture**:
  - Producer publishes, consumer processes asynchronously.

---

## ## 5) General Topics

### ### Project Information (How to present)

Use this format:

- Problem statement
- Your role and ownership
- Tech stack
- Challenges faced
- Solution approach
- Outcome/impact (performance, reliability, user benefit)

### ### Authentication vs Authorization

- **Authentication**: Who are you? (login)
- **Authorization**: What can you access? (roles/permissions)

### ### Microservices (Interview-ready points)

- Service boundaries by business domain
- Independent deployment/scaling
- API contracts and versioning
- Observability: logs, metrics, tracing

---

## ## 6) Programming Round – Sorting Array

### ### What interviewer may ask

- Sort array in ascending and descending order.
- Handle duplicates and negative numbers.
- Explain time complexity.

### ### Key explanation points

- Built-in sort needs comparator for numeric sort.
- Ascending comparator: smaller first.
- Descending comparator: bigger first.
- Complexity for standard comparison sort is usually  $O(n \log n)$ .

---

## ## 7) Last-Minute Interview Strategy (Very Important)

- Keep answers practical and based on your real work.
- If you don't know exact answer, explain known part honestly.
- Prefer structure: **definition → example → where used in project**.
- For every concept, prepare one short real example.
- Speak slowly and naturally; avoid memorized robotic lines.

---

## ## 8) Personalization Note

I could not find your resume file in this repository. If you share your resume text/PDF, I can create a **fully personalized self-introduction** (30 sec, 60 sec, and 90 sec versions) and project-specific interview answers.